

Project 1

AUTHOR
Grace Fischer

PUBLISHED
September 20, 2026

Setting Up

```
## Install the tidyverse package
if (!require("tidyverse", quietly = TRUE)) {
  install.packages("tidyverse")
}

library("tidyverse")

## Install the tidytuesdayR package
if (!require("tidytuesdayR", quietly = TRUE)) {
  install.packages("tidytuesdayR")
}

library("here")
library("tidyverse")

# tests if a directory named "data" exists locally
if (!dir.exists(here("data"))) {
  dir.create(here("data"))
}

# saves data only once (not each time you knit a R Markdown)
if (!file.exists(here("data", "chocolate.RDS"))) {
  url_csv <-
    "https://raw.githubusercontent.com/rfordatascience/tidy
    01-18/chocolate.csv"
  chocolate <- readr::read_csv(url_csv)

  # save the file to RDS objects
  saveRDS(chocolate, file = here("data", "chocolate.RDS"))
}

# Read dataset locally from computing environment
chocolate <- readRDS(here("data", "chocolate.RDS"))
as_tibble(chocolate)
```

```
# A tibble: 2,530 × 10
  ref company_manufacturer company_location review_date
  <dbl> <chr>                <chr>          <dbl>
1  2454 5150                U.S.A.          2019
2  2458 5150                U.S.A.          2019
3  2454 5150                U.S.A.          2019
4  2542 5150                U.S.A.          2021
5  2546 5150                U.S.A.          2021
6  2546 5150                U.S.A.          2021
7  2542 5150                U.S.A.          2021
8    797 A. Morin          France          2012
9    797 A. Morin          France          2012
10 1011 A. Morin          France          2013
# i 2,520 more rows
# i 6 more variables: country_of_bean_origin <chr>,
#   specific_bean_origin_or_bar_name <chr>, cocoa_percent
#   <chr>,
#   ingredients <chr>, most_memorable_characteristics <chr>,
#   rating <dbl>
```

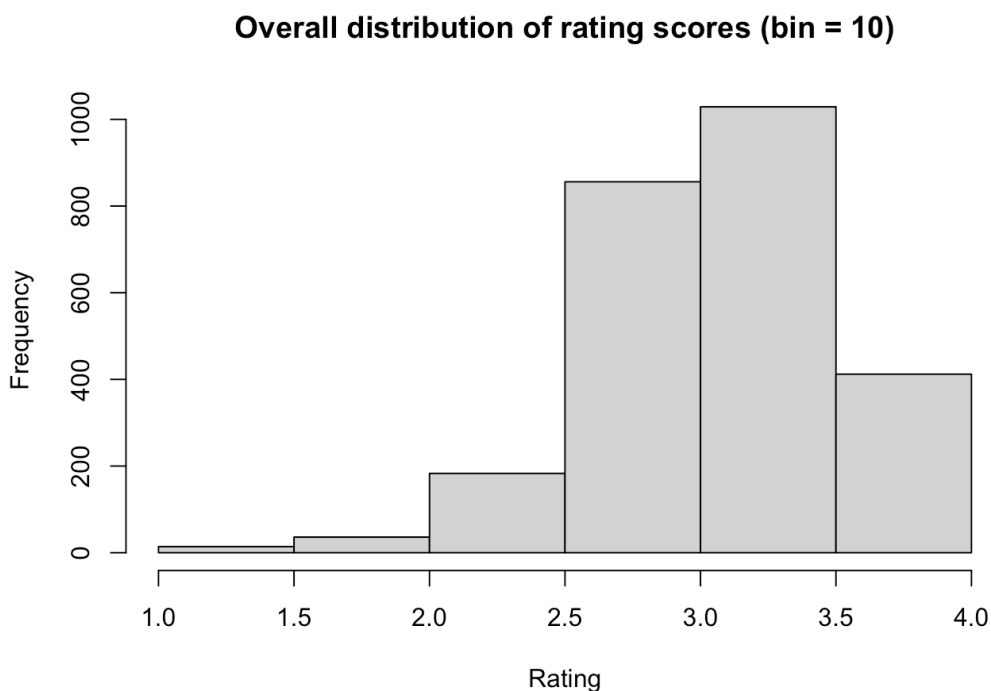
```
# Glimpse at the data
glimpse(chocolate)
```

```
Rows: 2,530
Columns: 10
$ ref                <dbl> 2454, 2458, 2454,
2542, 2546, 2546, 2...
$ company_manufacturer <chr> "5150", "5150",
"5150", "5150", "5150...
$ company_location    <chr> "U.S.A.", "U.S.A.",
"U.S.A.", "U.S.A....
$ review_date         <dbl> 2019, 2019, 2019,
2021, 2021, 2021, 2...
$ country_of_bean_origin <chr> "Tanzania", "Dominican
Republic", "Ma...
$ specific_bean_origin_or_bar_name <chr> "Kokoa Kamili, batch
1", "Zorzal, bat...
$ cocoa_percent       <chr> "76%", "76%", "76%",
"68%", "72%", "8...
$ ingredients         <chr> "3- B,S,C", "3-
B,S,C", "3- B,S,C", "...
$ most_memorable_characteristics <chr> "rich cocoa, fatty,
breathy", "cocoa, ...
$ rating              <dbl> 3.25, 3.50, 3.75,
```

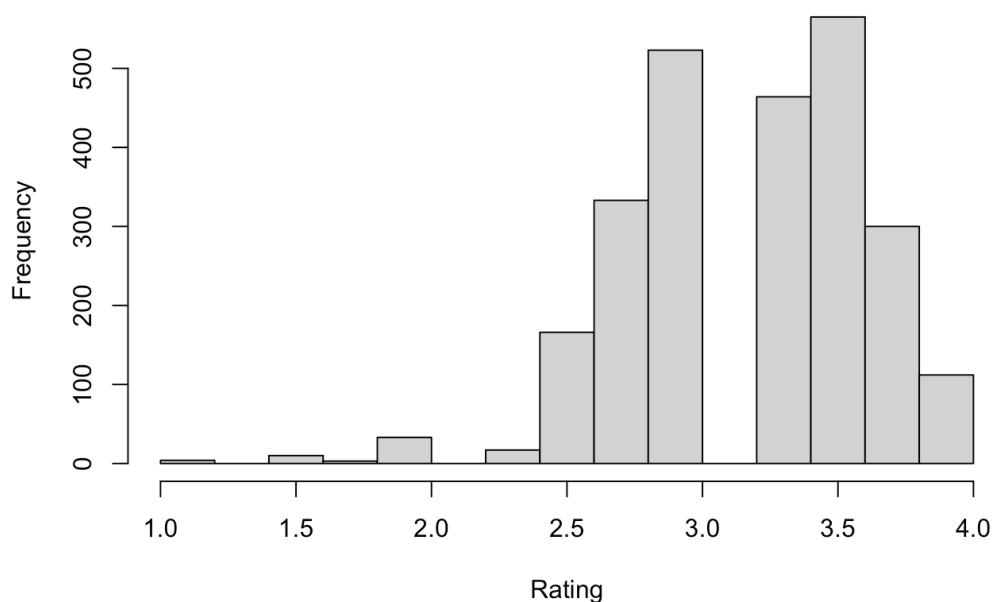
3.00, 3.00, 3.25, 3...

Part 1: Explore Data

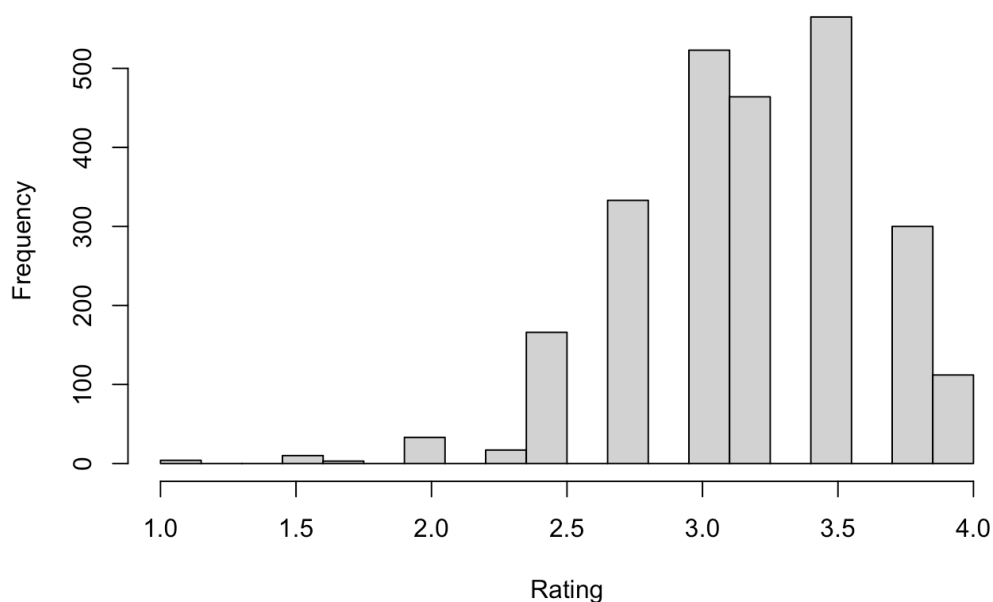
```
## Histogram where bin num = 10  
hist(chocolate$rating, main = "Overall distribution of rating  
scores (bin = 10)", xlab = "Rating", ylab =  
"Frequency", breaks = 10)
```



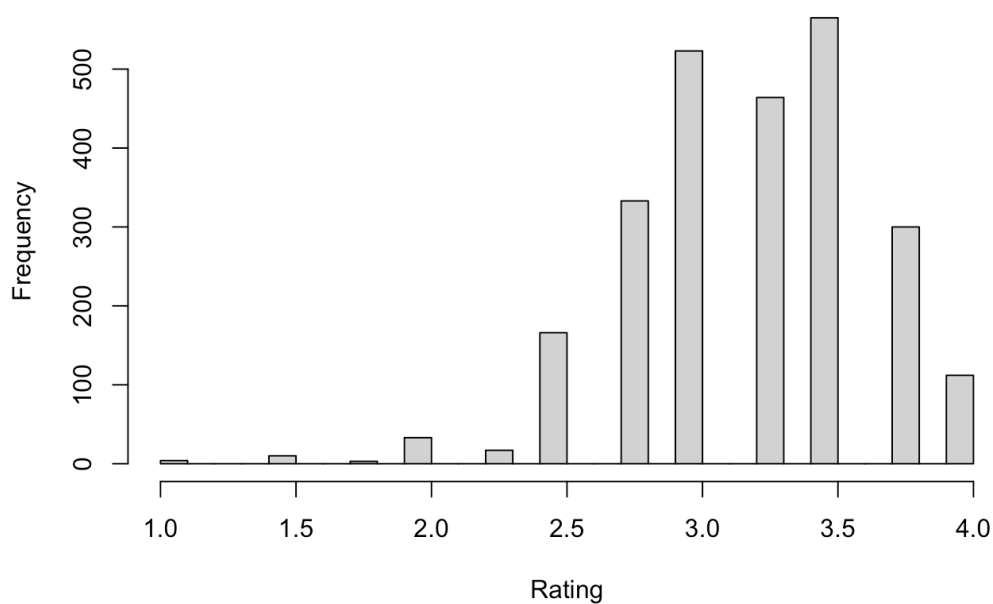
```
## Histogram where bin num = 15  
hist(chocolate$rating, main = "Overall distribution of rating  
scores (bin = 15)", xlab = "Rating", ylab =  
"Frequency", breaks = 15)
```

Overall distribution of rating scores (bin = 15)

```
## Histogram where bin num = 20 (breaks this way because r
    typically favors pretty numbers over round, length
    out is because it produces n-1 bins)
hist(chocolate$rating, main = "Overall distribution of rating
    scores (bin = 20)", xlab = "Rating", ylab =
    "Frequency", breaks = seq(min(chocolate$rating),
    max(chocolate$rating), length.out = 21))
```

Overall distribution of rating scores (bin = 20)

```
## Histogram where bin num = 25  
hist(chocolate$rating, main = "Overall distribution of rating  
scores (bin = 25)", xlab = "Rating", ylab =  
"Frequency", breaks = 25)
```

Overall distribution of rating scores (bin = 25)

The change that occurs when selecting a different number of bins is that the number of bars on the histogram change, when breaks = 10, there's 10 bars on the histogram across the x axis. When breaks = 15, there's 15 bars on the histogram across the x axis and so on. In my opinion, the one that looks the best is when breaks = 10. Typically, a histogram has all bars touching, and when we increase to bin = 15, the space between and roughly 3.3 drops to 0 or so small we cannot see it visually. The same problem occurs in bin = 20. When we make it to bin = 25, there is so much gapping between the bins, the histogram takes the shape of a bar chart, and loses its shape completely. Additionally, throughout all bin variations, the negative skew (or left skew) remains clear, so we are able to draw relatively the same conclusions like mean < median < mode. Though bin = 15 had advantages like providing more specific data, I think that what it adds does not outweigh the benefits of bin = 10.

```
## Arranging Data by the country of origin
arrange(chocolate, country_of_bean_origin)
```

```
# A tibble: 2,530 × 10
  ref company_manufacturer company_location review_date
  <dbl> <chr>                <chr>          <dbl>
1   785 Daintree          Australia      2011
2  1654 Daintree          Australia      2015
3  1864 Smooth Chocolator, The Australia      2016
4  1494 Ambrosia           Canada        2015
5  2586 Animas             U.S.A.        2021
6  1598 Arete              U.S.A.        2015
7  2290 Bankston           U.S.A.        2019
8   1768 Belyzium          Germany        2016
9   1768 Belyzium          Germany        2016
10  1768 Belyzium          Germany        2016
# i 2,520 more rows
# i 6 more variables: country_of_bean_origin <chr>,
#   specific_bean_origin_or_bar_name <chr>, cocoa_percent
#   <chr>,
#   ingredients <chr>, most_memorable_characteristics <chr>,
#   rating <dbl>
```

```
## Grouping by the country of origin
by_country <- group_by(chocolate, country_of_bean_origin)
```

```
## Summarizing data based on country, and the sum of the count
  of ratings.
country_and_review <- summarize(by_country,
  num_reviews = sum(!is.na(rating))
)
print(country_and_review, n =Inf)
```

A tibble: 62 × 2

	country_of_bean_origin	num_reviews
	<chr>	<int>
1	Australia	3
2	Belize	76
3	Blend	156
4	Bolivia	80
5	Brazil	78
6	Burma	1
7	Cameroon	3
8	China	1
9	Colombia	79
10	Congo	11
11	Costa Rica	43
12	Cuba	12
13	DR Congo	1
14	Dominican Republic	226
15	Ecuador	219
16	El Salvador	6
17	Fiji	16
18	Gabon	1
19	Ghana	41
20	Grenada	19
21	Guatemala	62
22	Haiti	30
23	Honduras	25
24	India	35
25	Indonesia	20
26	Ivory Coast	7
27	Jamaica	24
28	Liberia	3
29	Madagascar	177
30	Malaysia	8
31	Martinique	1
32	Mexico	55
33	Nicaragua	100
34	Nigeria	3
35	Panama	9

36	Papua New Guinea	50
37	Peru	244
38	Philippines	24
39	Principe	1
40	Puerto Rico	7
41	Samoa	3
42	Sao Tome	14
43	Sao Tome & Principe	2
44	Sierra Leone	4
45	Solomon Islands	10
46	Sri Lanka	2
47	St. Lucia	10
48	St.Vincent–Grenadines	1
49	Sulawesi	1
50	Sumatra	1
51	Suriname	1
52	Taiwan	2
53	Tanzania	79
54	Thailand	5
55	Tobago	2
56	Togo	3
57	Trinidad	42
58	U.S.A.	33
59	Uganda	19
60	Vanuatu	13
61	Venezuela	253
62	Vietnam	73

The amount of reviews that come from each country of origin are shown in the output from the code above.

```
## Filtering data for Ecuador
only_Ecuador <- filter(chocolate, country_of_bean_origin ==
  "Ecuador")

## Calc mean of ratings from new data frame
avg_rating_ecu <- mean(only_Ecuador$rating)
avg_rating_ecu
```

```
[1] 3.164384
```

```
## Print the average
```



```
print(paste("The average rating of chocolate bars where the
            country of origin of beans is Ecuador is: ",
            avg_rating_ecu))
```

```
[1] "The average rating of chocolate bars where the country of
origin of beans is Ecuador is: 3.16438356164384"
```

```
## Creating Ecuador Summary
ecuador_summary <- only_Ecuador %>%
  summarize (
    total = sum(!is.na(rating)),
    mean = mean(rating),
    sd = sd(rating),
  )
ecuador_summary
```

```
# A tibble: 1 × 3
  total mean    sd
  <int> <dbl> <dbl>
1   219  3.16 0.512
```

```
## Find location with Best Chocolate
best_location <- only_Ecuador %>%

## Group by the company location
group_by(company_location) %>%

## Calculate the average rating
summarize(
  mean_rating = mean(rating, na.rm = TRUE),
) %>%

## Sort from highest average rating to lowest
arrange(desc(mean_rating))

# Print locations
print(best_location, n = Inf)
```

```
# A tibble: 25 × 2
  company_location mean_rating
  <chr>             <dbl>
1 Australia         3.81
2 Switzerland       3.75
3 New Zealand       3.62
```

4	Hungary	3.5
5	Israel	3.5
6	Netherlands	3.5
7	Singapore	3.5
8	South Korea	3.33
9	Canada	3.28
10	U.S.A.	3.26
11	Argentina	3.25
12	Nicaragua	3.25
13	Poland	3.25
14	Russia	3.25
15	France	3.19
16	Italy	3.19
17	U.K.	3.08
18	Belgium	3.05
19	Ecuador	3.00
20	Spain	2.94
21	Austria	2.88
22	Denmark	2.5
23	Scotland	2.5
24	South Africa	2.5
25	Germany	2.44

The company location with the best average ratings with beans from Ecuador is Australia.

```
##
## Find countries of origin with best average ratings
best_overall_ratings <- chocolate %>%

## Group by the country of origin
group_by(country_of_bean_origin) %>%

## Calculate the average rating
summarize(
  mean_rating = mean(rating, na.rm = TRUE),
) %>%

## Sort from highest average rating to lowest
arrange(desc(mean_rating))

# Print locations
print(head(best_overall_ratings, 3))
```

```
# A tibble: 3 × 2
  country_of_bean_origin mean_rating
  <chr>                  <dbl>
1 Tobago                 3.62
2 China                  3.5
3 Sao Tome & Principe    3.5
```

The countries of bean origin with the best average ratings are Tobago with 3.62, China with 3.5, and Sao Tome & Principe with 3.5.

```
## Find countries of origin with best average ratings
best_ratings_10 <- chocolate %>%

## Group by the country of origin
group_by(country_of_bean_origin) %>%

## Calculate the average rating
summarize(
  avg_rating = mean(rating, na.rm = TRUE),
  num_reviews = sum(!is.na(rating))
) %>%

## Filter out countries with less than 10 reviews
filter(num_reviews >= 10) %>%

## Sort from highest average rating to lowest
arrange(desc(avg_rating))

# Print locations
print(head(best_ratings_10, 3))
```

```
# A tibble: 3 × 3
  country_of_bean_origin avg_rating num_reviews
  <chr>                  <dbl>      <int>
1 Solomon Islands       3.45         10
2 Congo                 3.32         11
3 Cuba                  3.29         12
```

Now, filtering for 10 reviews or more, the countries of bean origin with the best average ratings are Solomon Islands with 3.45 rating and 10 reviews, Congo with 3.32 rating and 11 reviews, and Cuba with 3.29 rating and 12 reviews.

```
## Identify the countries of bean origin with at least 50
  reviews.
new_choco <- chocolate %>%

## Filter out countries with less than 50 reviews for country
  of origin
  group_by(country_of_bean_origin) %>%
  filter(sum(!is.na(rating)) >= 50)

print(new_choco)
```

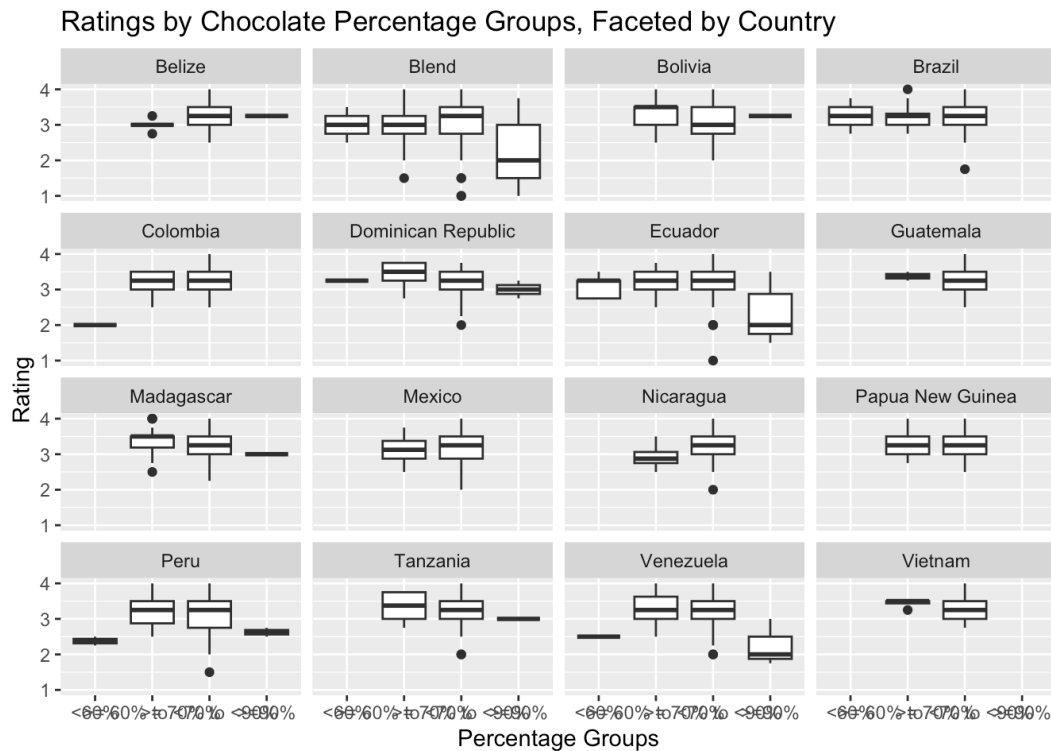
```
# A tibble: 2,007 × 10
# Groups:   country_of_bean_origin [16]
  ref company_manufacturer company_location review_date
  <dbl> <chr>                <chr>          <dbl>
1  2454 5150                U.S.A.          2019
2  2458 5150                U.S.A.          2019
3  2454 5150                U.S.A.          2019
4  2546 5150                U.S.A.          2021
5    797 A. Morin          France          2012
6    797 A. Morin          France          2012
7   1015 A. Morin          France          2013
8   1011 A. Morin          France          2013
9   1011 A. Morin          France          2013
10  1015 A. Morin          France          2013
# i 1,997 more rows
# i 6 more variables: country_of_bean_origin <chr>,
#   specific_bean_origin_or_bar_name <chr>, cocoa_percent
<chr>,
#   ingredients <chr>, most_memorable_characteristics <chr>,
rating <dbl>
```

```
## Using mutate function to add a new column
new_choco <- new_choco %>%
  mutate(

    ## Converting columns to numbers using substr and nchar -
      1 for indexing str
    cocoa_percent = as.numeric(substr(cocoa_percent, 1,
      nchar(cocoa_percent) - 1)),

    ## Using case when to break up mutate var
    percent_groups = case_when(
      cocoa_percent < 60 ~ "<60%",
```

```
cocoa_percent >= 60 & cocoa_percent < 70 ~ ">= 60% to  
  <70%",  
cocoa_percent >= 70 & cocoa_percent < 90 ~ ">= 70% to  
  <90%",  
cocoa_percent >= 90 ~ ">=90%"  
)  
)  
  
## re-order factor levels (just in case and for practice).  
new_choco <- new_choco %>%  
  
## ungrouping to avoid error  
ungroup()%>%  
  
## implementing mutate and fct_relevel  
mutate(  
  percent_groups = fct_relevel(percent_groups, "<60%", ">=  
    60% to <70%", ">= 70% to <90%", ">=90%")  
)  
  
## Use ggplot to create boxplots, use aes to set x and y  
ggplot(new_choco, aes(x=percent_groups, y = rating)) +  
  ## Using geom.boxplot  
  geom_boxplot() +  
  
  ## this gets side by side plots (see source at end)  
  facet_wrap( ~ country_of_bean_origin) +  
  
  ## (labels for graph)  
  labs( x = "Percentage Groups",  
    y = "Rating",  
    title = "Ratings by Chocolate Percentage Groups, Faceted  
      by Country")
```



On average, which category of chocolate percentage is most highly rated?
Do these countries mostly agree or are there disagreements?

On average, I think that ">= 60% to <70%" and the ">= 70% to <90%" groups are the groups that were the most highly rated, since both groups consistently have a high median category. It is hard to say because favor one group over the other, but both are ranked on the higher end a majority of the time, one is not consistently higher than the other, and oftentimes they are tied. Countries tend to agree that the >=90% group rates consistently low.

Sources I used to help me answer this question:

<https://stackoverflow.com/questions/49458591/use-facet-wrap-in-r>

I was not familiar with this, so it was fun to learn about this way of using facet-wraps. I remember a different way of doing it from undergrad (I was a Statistics major, compsci minor), so this was actually quite interesting.

Part 2: Join two datasets together

```
## Installing gapminder
```

```
## install.packages("gapminder") already installed
library("gapminder")

## Setting up continent before joining
country_by_cont <- gapminder::gapminder_unfiltered %>%
  distinct(country, continent)

## Left join to keep row in chocolate and add continent
chocolate <- chocolate %>%
  ## Making check for countries
  left_join(country_by_cont, by = c("country_of_bean_origin" =
    "country"))
```

```
chocolate <- chocolate %>%

## Group by the country of origin
group_by(country_of_bean_origin) %>%

## Filter out countries with less than 10 reviews
filter(sum(!is.na(rating)) >= 10) %>%

## ungrouping to avoid error
ungroup()
```

```
chocolate <- chocolate %>%

## Filter out countries with Blend
filter(country_of_bean_origin != "Blend")
```

```
## I waited to do this before graphs since we were removing
   countries with less than 10 reviews, and blend
   anyway!
```

```
## Checking to see when a continent is NA
chocolate %>%
  filter(is.na(continent)) %>%
  distinct(country_of_bean_origin)
```

```
# A tibble: 5 × 1
  country_of_bean_origin
  <chr>
1 Sao Tome
2 Trinidad
```

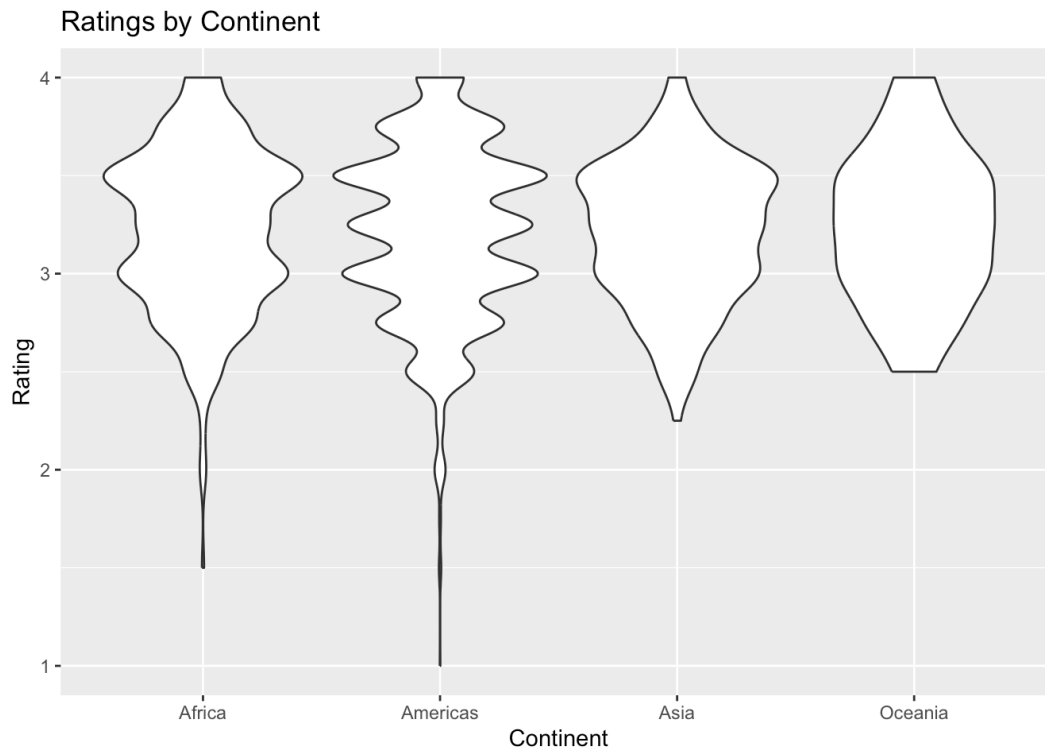
3 Congo
 4 St. Lucia
 5 U.S.A.

```
## Fixing NA continents listed
chocolate <- chocolate %>%
  mutate(
    ## Using case when to prepare functions
    continent = case_when(
      country_of_bean_origin %in% c("Sao Tome", "Congo") ~
        "Africa",
      country_of_bean_origin %in% c("Trinidad", "St. Lucia",
        "U.S.A.") ~ "Americas",
      ## Need this for rows that already assigned
      TRUE ~ continent
    )
  )

## Run check again to make sure all countries have continents
chocolate %>%
  filter(is.na(continent)) %>%
  distinct(country_of_bean_origin)
```

A tibble: 0 × 1
 # i 1 variable: country_of_bean_origin <chr>

```
## Using ggplot and aes, setting x and y respectively
ggplot(chocolate, aes(x = continent, y = rating)) +
  geom_violin() +
  labs(
    x = "Continent",
    y = "Rating",
    title = "Ratings by Continent")
```

Part 3: Convert wide data into long data

```
chocolate <- chocolate %>%

## Accounting for NA's which would go undetected, mutate acts
## as boolean here and does the 0, 1's
mutate(
  beans      =
    replace_na(as.integer(str_detect(ingredients, "B")),
    0),
  sugar      =
    replace_na(as.integer(str_detect(ingredients,
    "S,|S$")), 0),
  cocoa_butter =
    replace_na(as.integer(str_detect(ingredients, "C")),
    0),
  vanilla    =
    replace_na(as.integer(str_detect(ingredients, "V")),
    0),
  letchin    =
    replace_na(as.integer(str_detect(ingredients, "L")),
    0),
```

```

    salt      =
      replace_na(as.integer(str_detect(ingredients, "Sa")),
        0)
  )

```

Checking for ket words in most memorable characteritsics, here the muatate function acting as boolean, assign 0 is word not detected.

```

chocolate <- chocolate %>%
  mutate(
    char_cocoa =
      as.integer(str_detect(most_memorable_characteristics,
        "cocoa")),
    char_sweet =
      as.integer(str_detect(most_memorable_characteristics,
        "sweet")),
    char_nutty =
      as.integer(str_detect(most_memorable_characteristics,
        "nutty")),
    char_creamy =
      as.integer(str_detect(most_memorable_characteristics,
        "creamy")),
    char_roasty =
      as.integer(str_detect(most_memorable_characteristics,
        "roasty")),
    char_earthy =
      as.integer(str_detect(most_memorable_characteristics,
        "earthy"))
  )

```

Calculate means for columns

```

chocolate <- chocolate %>%
  ## Group by review year
  group_by(review_date) %>%
  ## Summarize the means
  summarize(
    beans      = mean(beans),
    sugar      = mean(sugar),
    cocoa_butter = mean(cocoa_butter),
    vanilla    = mean(vanilla),
    letchin    = mean(letchin),
    salt       = mean(salt),
    char_cocoa = mean(char_cocoa),
    char_sweet = mean(char_sweet),
    char_nutty = mean(char_nutty),
    char_creamy = mean(char_creamy),
    char_roasty = mean(char_roasty),
  )

```

```

    char_earthly = mean(char_earthly),
  )

## Check for specs: 16 rows 13 cols
chocolate

```

```
# A tibble: 16 × 13
```

```

  review_date beans sugar cocoa_butter vanilla letchin salt
char_cocoa

```

```

      <dbl> <dbl> <dbl>          <dbl>  <dbl>  <dbl>  <dbl>
<dbl>

```

```

  1      2006 1      1      0.925 0.7      0.725 0
0.175
  2      2007 0.930 0.930      0.702 0.456      0.281 0
0.351
  3      2008 0.915 0.902      0.780 0.378      0.524 0
0.110
  4      2009 0.915 0.915      0.755 0.321      0.302 0
0.142
  5      2010 0.897 0.887      0.753 0.227      0.412 0.0103
0.175
  6      2011 0.944 0.916      0.671 0.0979      0.119 0.0559
0.154
  7      2012 0.930 0.837      0.669 0.180      0.122 0.0581
0.0872
  8      2013 0.964 0.946      0.760 0.204      0.275 0.0180
0.174
  9      2014 0.987 0.947      0.645 0.0702      0.127 0.0307
0.0570
 10      2015 0.985 0.933      0.541 0.0522      0.104 0
0.127
 11      2016 0.981 0.961      0.607 0.0437      0.102 0.00971
0.0922
 12      2017 0.98  0.97      0.55  0.03      0.13  0.01
0.14
 13      2018 0.986 0.933      0.577 0.0625      0.130 0
0.178
 14      2019 1      0.972      0.672 0.0282      0.186 0
0.266
 15      2020 1      1      0.566 0.0395      0.0263 0
0.276
 16      2021 1      1      0.636 0.00606      0.0727 0
0.285

```

```
# i 5 more variables: char_sweet <dbl>, char_nutty <dbl>,
```

```
char_creamy <dbl>,
# char_roasty <dbl>, char_earthy <dbl>
```

```
## Sources I used to help me included at end
chocolate_long <- chocolate %>%
  pivot_longer(
    cols = beans:char_earthy,
    names_to = "feature",
    values_to = "mean_score"
  )

chocolate_long
```

```
# A tibble: 192 × 3
  review_date feature      mean_score
      <dbl> <chr>          <dbl>
1      2006 beans            1
2      2006 sugar            1
3      2006 cocoa_butter    0.925
4      2006 vanilla         0.7
5      2006 letchin        0.725
6      2006 salt            0
7      2006 char_cocoa     0.175
8      2006 char_sweet     0.175
9      2006 char_nutty     0.05
10     2006 char_creamy     0.375
# i 182 more rows
```

Sources I used to help me complete this:

<https://stackoverflow.com/questions/72482406/how-to-use-pivot-longer-as-a-newbie>

Part 4: Data Visualization

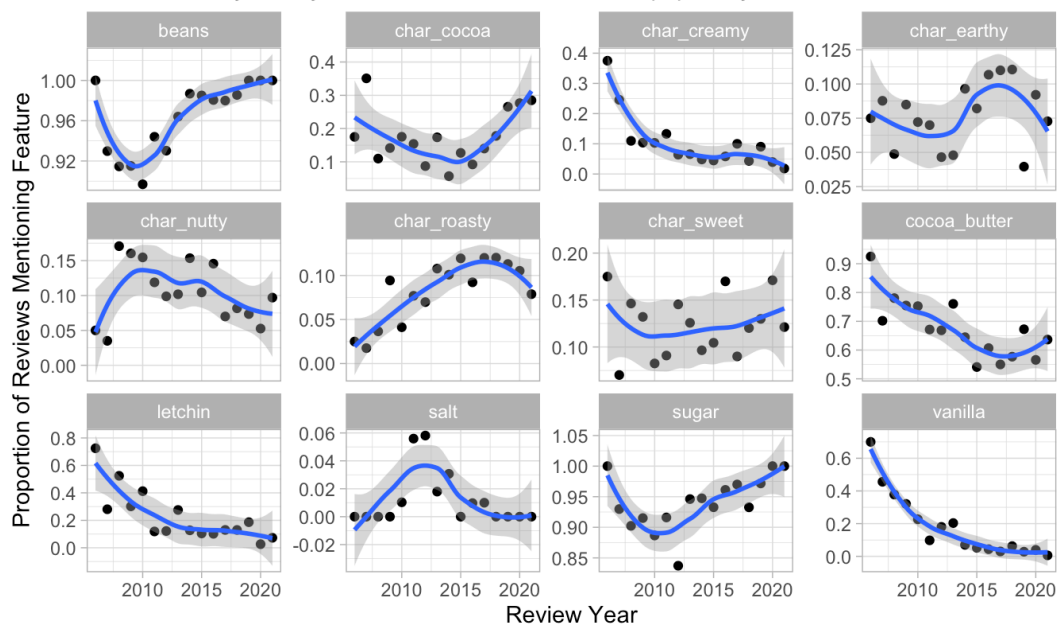
Using chocolate_long from Part 3

```
## Use ggplot with data and aes for x and y respectively
ggplot(chocolate_long, aes(x = review_date, y = mean_score)) +
  ## Plot type
  geom_point() +
  ## 2. non-linear line (mean points above by geom.point)
  geom_smooth(method = "loess" ) +
```

```
## 3. all plots shown in 1 figure, with individual scales
      for numbers depending on data "free_y" (source below)
facet_wrap(~ feature, scales = "free_y") +
labs(
  ## 4. x and y axis labels
  x = "Review Year",
  y = "Proportion of Reviews Mentioning Feature",
  ## 1. Title, subtitle, caption
  title = "Chocolate Ingredients and Flavor Variations
          Through the Years",
  subtitle = "Since around 2006, there has been a decline
             notes such as vanilla, lecithin, and cream, \n
             whereas earthy, roasty, and cocoa notes have risen in
             popularity.",
  caption = "Grace Fischer") +
theme_light()
```

Chocolate Ingredients and Flavor Variations Through the Years

Since around 2006, there has been a decline notes such as vanilla, lecithin, and cream, whereas earthy, roasty, and cocoa notes have risen in popularity.



Grace Fischer

Sources I used to help me complete this:

<https://stackoverflow.com/questions/18046051/setting-individual-axis-limits-with-facet-wrap-and-scales-free-in-ggplot2>

Part 5: Make the worst plot you can

```

## For this part, I will be using ggplot chocolate with all of
  the data from the data set, using aes to set x and y
  respectively

## 1. For my first modification I am going to choose the color
  of the data, based on which country the bean belong
  to, it will be colored accordingly.

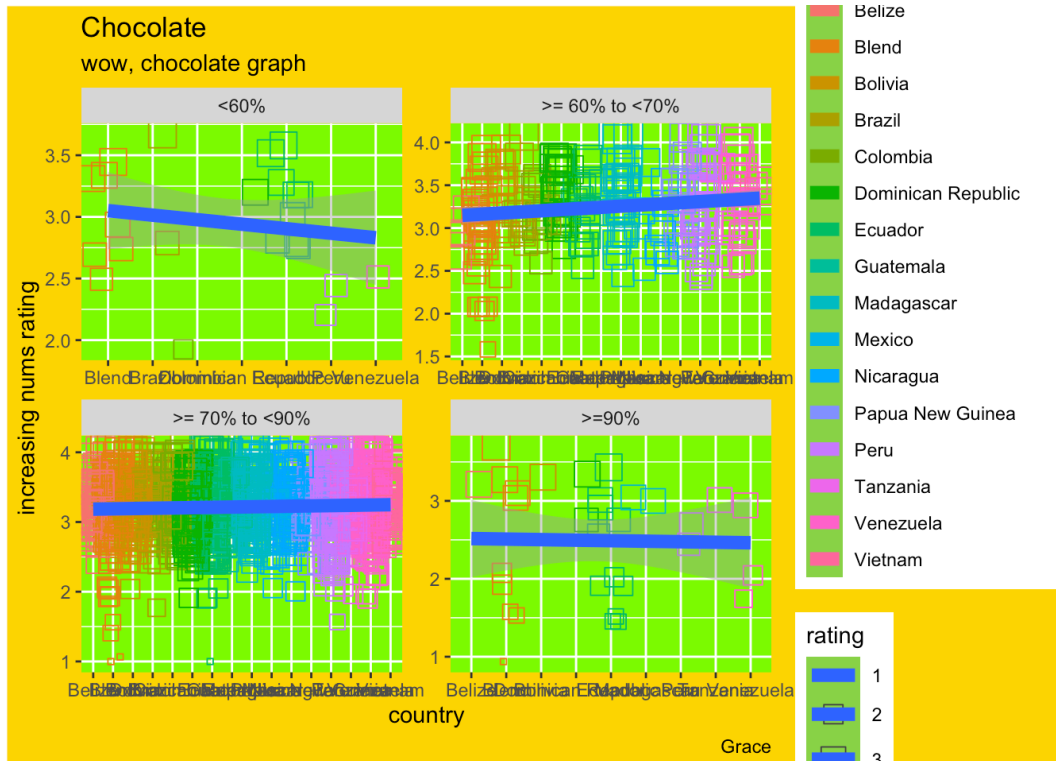
## 2. My second modification is sizing by rating. This way,
  the scale of the dot will be based upon the rating
  value.
ggplot(new_choco, aes(x = country_of_bean_origin, y = rating,
  color = country_of_bean_origin, size = rating)) +

## 3. The third modification is using geom_jitter() which
  offsets points so they are not overlapping (in this
  case since countries are the same, there would be
  some overlapping)
## 4. My fourth modification is using shape = 0 which makes
  the shape of the points a square that is hollow
geom_jitter(shape = 0) +
  ## 5. My fifth modification is fitting an lm line which is
  not necessarily good since country of origin is a
  categorical and not numerical variable
  ## 6. I also chose to make the linewidth 3, this way it is
  abnormally thick and draws attention away from data
  points.
geom_smooth(aes(group = 1), method = "lm", linewidth = 3) +

##7. I broke up the graphs by the percent_group scales and
  now

facet_wrap(~percent_groups, scales = "free" ) +
  ## 8. I added labels to the graphs and made them bad
  just to make the worst graph
labs(
  x = "country",
  y = "increasing nums rating",
  title = "Chocolate",
  subtitle = "wow, chocolate graph",
  caption = "Grace"
) +
  ## 9. I made the plot background a different color
  theme(
    plot.background = element_rect(fill = "gold"),
    panel.background = element_rect(fill = "chartreuse")
  )

```



Sources I used to help me complete this:

https://ggplot2.tidyverse.org/reference/geom_jitter.html

1. Creating link between color to country_of_bean_origin here is not good because with so many countries it is hard to memorize, however, this could be good to use in in a visualization with a low number of categories, to keep the number of colors low and easy to remember, making it distinct.
2. Sizing by rating in this graph is slightly redundant since rating is shown on y axis, however, in a good visualization using a different variable would make it more informative and useful.
3. Using geom_jitter allows overlapping points to not be on top of one another so they can be seen better, but here there is a heavy separation because of the data so it can misrepresent the data, making it look more spread out than it really is. In a good visualization, when used carefully, it can be useful.
4. Changing the shape of the points can be useful in a good visualization, like using high contrast shapes that are easy to read, even changing them to represent different categories. Here to make a bad graph I chose shapes that were intentionally difficult to read and would overlap because of aforementioned modifications, to prove my point.
5. Fitting the lm line in this case makes the slope meaningless since it is

categorical. In a good visualization, a trendline would fit the data and use loess or whatever shape would meet it. I had to add the aes line to make it show up.

6. Making the line as thick as it is draws attention away from the data points where the line is suppose to enhance the data, not be a focal point. In a good graph, it would be subtle and help guide the viewer across the data, not distract.
7. Faceting percent groups an using scale = free lets the axis range be independent on each of the panels. This makes it easy to misjudge, since they are all measuring the same thing. To use this in a good visualization, all scales would be the same or scale = free would be used when the range is unrelated to the others.
8. This is not much of a modification but i felt it should be included. The use of labels, titles, subtitles, and captions should be meaningful in a good visualization so the viewer can figure out what is going on.
9. Using panel and plot colors that are clashing is an eye-sore and distracting. In a good visualization, we can use differeny colors to make it interesting, but still neutral and low contrast so that the data remains the focal point, and easy to read.

Part 6: Make my plot a better plot!

```
library("tidyverse")
chocolate <- readRDS(here("data", "chocolate.RDS"))
as_tibble(chocolate)
```

```
# A tibble: 2,530 × 10
```

	ref	company_manufacturer	company_location	review_date
	<dbl>	<chr>	<chr>	<dbl>
1	2454	5150	U.S.A.	2019
2	2458	5150	U.S.A.	2019
3	2454	5150	U.S.A.	2019
4	2542	5150	U.S.A.	2021
5	2546	5150	U.S.A.	2021
6	2546	5150	U.S.A.	2021
7	2542	5150	U.S.A.	2021
8	797	A. Morin	France	2012
9	797	A. Morin	France	2012
10	1011	A. Morin	France	2013

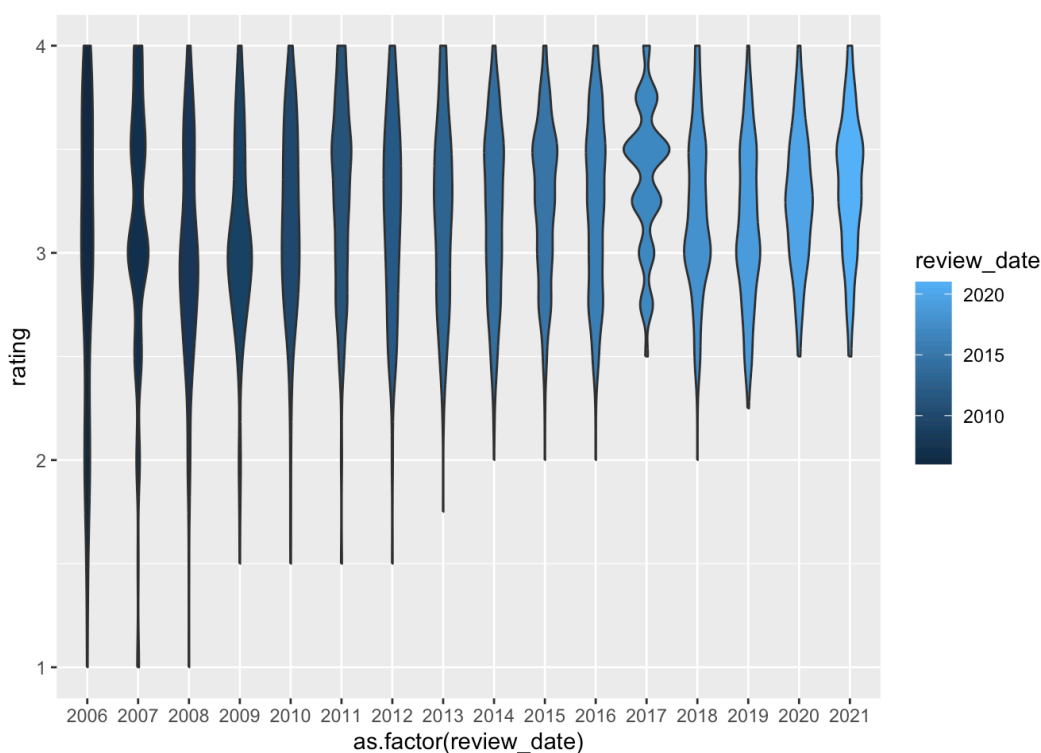
```
# i 2,520 more rows
```

```
# i 6 more variables: country_of_bean_origin <chr>,
```



```
# specific_bean_origin_or_bar_name <chr>, cocoa_percent
<chr>,
# ingredients <chr>, most_memorable_characteristics <chr>,
rating <dbl>
```

```
## Original plot
chocolate %>%
  ggplot(aes(
    x = as.factor(review_date),
    y = rating,
    fill = review_date
  )) +
  geom_violin()
```



```
## Improved Plot
chocolate %>%
  ggplot(aes(
    x = as.factor(review_date),
    y = rating,
  )) +
  ## 1. Using one color, the color distinction does not
  ## add anything, as year is already expressed on x axis,
  ## it is just redundant.
```

```

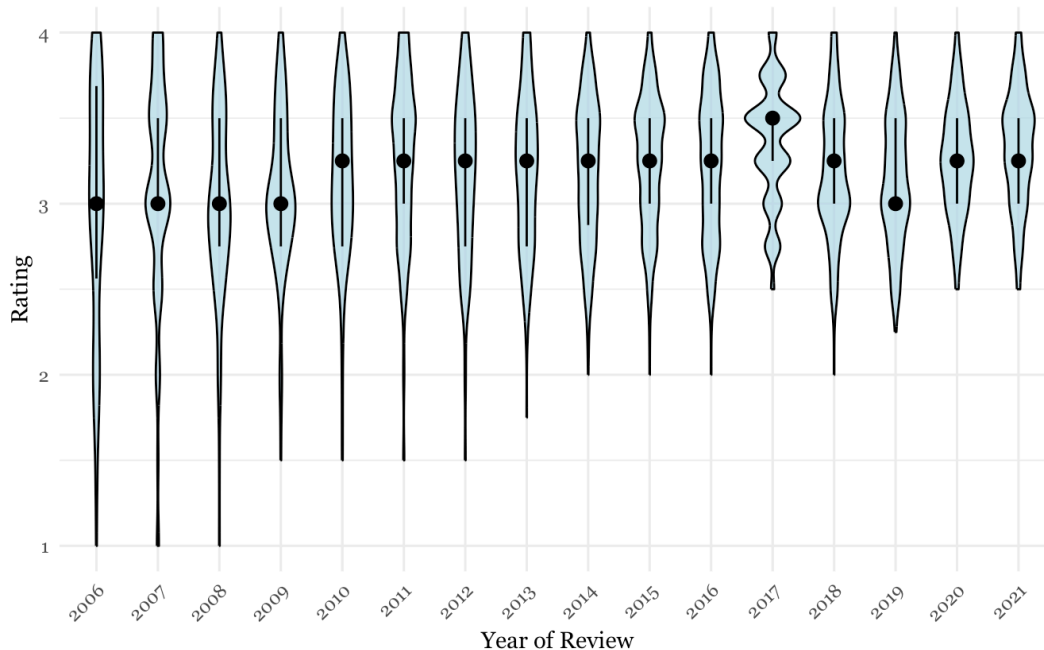
    ## 2. alpha to color so its not as opaque
    geom_violin(fill = "lightblue", color = "black", alpha =
      0.7) +
    ## 3. Added stat summary to show where Q1, median and Q3
      are. I added source below.
    stat_summary(
      fun.min = function(x) quantile(x, 0.25),
      fun.max = function(x) quantile(x, 0.75),
      fun = median,
      geom = "pointrange"
    ) +
      ## 4. Added this since points go 1-4, helps with white
        space
      coord_cartesian(ylim = c(1, 4)) +
      ## 5. Added labs to make graph more specific
    labs(
      x = "Year of Review",
      y = "Rating",
      title = "Chocolate Bar Ratings by Year",
      subtitle = "Rating distributions have remained fairly
        constant, with a slight upward\nshift as time goes
        on"
    ) +

    ## 6. Added a theme for consistensy and ease of reading.
    ## 7. Added base font
    theme_minimal(base_family = "Georgia") +
    ## 8. Adjusted fonts, angle, etc for clear reading, source
      below
    theme(
      plot.title = element_text(face = "bold", size = 15),
      plot.subtitle = element_text(color = "darkgray", size =
        10),
      axis.text.x = element_text(angle = 45, hjust = 1)
    )

```

Chocolate Bar Ratings by Year

Rating distributions have remained fairly constant, with a slight upward shift as time goes on



Sources I used to help me complete this:

<https://stackoverflow.com/questions/41077199/how-to-use-r-ggplot-stat-summary-to-plot-median-and-quartiles>

<https://stackoverflow.com/questions/1330989/rotating-and-spacing-axis-labels-in-ggplot2>

1. I removed the fill = review date. In the original plot it was displaying a variable that was already being shown on the plot. It displaying an unnecessary gradient that did not provide any additional information to the viewer, so instead I made it a single color instead of the gradient with a key.
2. I added alpha to the fill. After doing teh fill I thought the colors I was trying all came out too opaque and heavy. This adds softness and an element of transparency to the graph, overall it makes it look more polished.
3. I added stat_summary. Here, it displays Q1, Q2 (median), and Q3. Though the violin shows the overall distribution shape, it has no summary or specific statistics. I felt that this gave the viewer the exact IQR and median qithout the entire boxplot making it look to busy or heavy.
4. I used coord_cartesian to do y-axis range. It only goes from 1-4, so this removes any uncessary white space. Though there was not much

to begin with, incase there would be after any edits this got tid of that and also keeps violin shapes and quartile calculations accurate.

5. I added labels to the graph to help the viewer better understand what they are looking at. The original plot had no labels at all, and even left the x label as what it is in the code. Adding this helps the viewer understand what they are looking at.
6. I added the minimal theme. The default theme has a heavy visual weight compared to the violin graphs, this filters out some of the background additions it had before.
7. I changed the font to Georgia. The font before felt plain and changing to a serif font felt more finished and clean to me
8. I customized the bold, color, and angle of the labels. Thsi way the viewer can distinguish whagt to read first. Additionally, angling the years prevents them from overlapping one another and potentially being unreadable at any point.